Memoria técnica de arquitectura de seguridad y control de acceso

Proyecto: Gestión de inventario y pedidos (Actividad individual 1) · Ionic 9 + Angular 22 standalone + Supabase
Alumno: Guillermo Prat Astillero · DAM 2 · Desarrollo de Interfaces (0488) · 30/10/2026 **Repositorio:** https://github.com/clax-dev/DI-gestion-inventario

> Modelo elegido: **RBAC con permisos** (los roles agrupan permisos; la interfaz pregunta por permisos, nunca por roles). Dos roles, tal como exige la actividad: **Administrador** y **Usuario estándar**.

A. Fundamentación teórico-técnica

A.1 Functional Guards (Angular standalone)

Un guard es una **función pura** (`CanActivateFn`, `CanMatchFn`, `CanDeactivateFn`) que el `Router` ejecuta durante la navegación y que devuelve `true`, `false` o un `UrlTree`. Se usa la API funcional con `inject()`; las clases `implements CanActivate` están obsoletas desde Angular 15. Devolver un `UrlTree` en vez de `false` + `router.navigate()` produce una **redirección atómica**: la navegación original se cancela y la nueva se programa en el mismo ciclo, sin parpadeos ni condiciones de carrera.

Orden en el ciclo de navegación de una ruta con `loadComponent`:

```
URL → 1. canMatch (¿encaja esta ruta?) → 2. descarga del chunk lazy → 3. canActivate (¿puede entrar?)
    → 4. resolvers → 5. activación del componente ····· al salir: canDeactivate (¿puede irse?)
```

Tipo	Momento	Uso en el proyecto
`canMatch`	Al **emparejar** la URL con la configuración	Pantallas restringidas por permiso (dashboard, alta de productos)
`canActivate`	Tras emparejar y cargar, antes de activar	Pantallas que solo requieren sesión (`authGuard`)
`canDeactivate`	Al **abandonar** la vista	Formulario de producto con cambios sin guardar

canActivate frente a canMatch en lazy loading. Con `canActivate` el router ya ha elegido la ruta y **descarga el chunk** del componente antes de evaluar el guard: un usuario sin permiso recibe igualmente el JavaScript de la pantalla de administración (coste de red y exposición de código). Con `canMatch`, si el guard falla la ruta "no existe" para ese usuario: el router continúa con la siguiente ruta coincidente y **el chunk nunca se descarga**. Por eso las rutas restringidas usan `canMatch`.

Consecuencia de diseño: se elimina `withPreloading(PreloadAllModules)` de `main.ts`, porque el precargador descarga en segundo plano todos los `loadComponent` sin evaluar `canMatch`, anulando su ventaja.

A.2 HTTP Interceptors (`HttpInterceptorFn`)

Un interceptor funcional `(req, next) => Observable<HttpEvent>` es un eslabón de una **cadena de responsabilidad**: cada petición de `HttpClient` atraviesa los interceptores en el orden de registro y las respuestas vuelven en orden inverso. `HttpRequest` es **inmutable**, por lo que para añadir cabeceras se usa `req.clone({ setHeaders })`.

- **Propagación del token:** el interceptor añade `Authorization: Bearer <access_token>` (JWT emitido por Supabase Auth) y la cabecera `apikey`, **solo** si la URL pertenece a nuestra API (`environment.supabaseUrl`), para no filtrar el token a terceros.
- **Errores centralizados** con `catchError`:
  - **401 Unauthorized** (token ausente/expirado): se intenta **una única vez** `supabase.auth.refreshSession()` (el *refresh token* es de larga duración y canjea un *access token* nuevo); si funciona, se **reintenta la petición original** con el nuevo token (`switchMap`); si falla, `logout()` y redirección a `/login?returnUrl=<url actual>`.
  - **403 Forbidden** (autenticado sin privilegio): no se cierra sesión; se muestra un aviso y se redirige al inicio del usuario.
- El error se re-emite (`throwError`) para que el componente pueda reaccionar (p. ej. ocultar el *spinner*).

Nota: el cliente `supabase-js` gestiona su propia cabecera y su refresco automático; el interceptor cubre las peticiones REST hechas con `HttpClient` (consumo SQL de la Fase 3), que es donde se centraliza la política.

A.3 RBAC

Modelo formal (NIST RBAC0): Usuarios U, Roles R, Permisos P; asignación $UA \subseteq U \times R$ y $PA \subseteq R \times P$. Un usuario puede realizar una acción si alguno de sus roles tiene el permiso. Los permisos se expresan como `recurso:acción`.

	Autenticación	**Autorización**
Pregunta	¿Quién eres?	¿Qué puedes hacer?
Mecanismo	Supabase Auth (email + contraseña) → JWT firmado	Mapa rol → permisos + guards + RLS
Fallo	401	403

Encapsulamiento del token. El JWT de Supabase contiene los *claims* `sub` (id de usuario), `exp` (expiración) y `role` (rol de **PostgreSQL**, siempre `authenticated`, no es el rol de negocio). El rol de la aplicación se añade como *claim* propio `app_rol` mediante un **Custom Access Token Hook** que lo lee de la tabla `perfiles`. Ningún componente lee el token: lo hace un único servicio de estado, `AuthStore`, que expone signals de solo lectura:

```
export type Rol = 'admin' | 'usuario';
export type Permiso = 'productos:ver' | 'productos:crear' | 'productos:editar'
  | 'pedidos:crear' | 'pedidos:ver_propios' | 'pedidos:ver_todos'
  | 'dashboard:ver' | 'informes:exportar' | 'actividad:ver';

/** PA ⊆ R × P: única fuente de verdad de los privilegios. */
export const PERMISOS_POR_ROL: Record<Rol, readonly Permiso[]> = {
  admin:   ['productos:ver', 'productos:crear', 'productos:editar', 'pedidos:ver_todos',
            'dashboard:ver', 'informes:exportar', 'actividad:ver'],
  usuario: ['productos:ver', 'pedidos:crear', 'pedidos:ver_propios'],
};

@Injectable({ providedIn: 'root' })
export class AuthStore {
  private readonly _session = signal<Session | null>(null);
  readonly isAuthenticated = computed(() => this._session() !== null);
  readonly rol = computed<Rol | null>(() => this.claims()?.app_rol ?? null);
  readonly permisos = computed(() => new Set(this.rol() ? PERMISOS_POR_ROL[this.rol()!] : []));
  can(p: Permiso): boolean { return this.permisos().has(p); }
  // claims() decodifica el payload del JWT; se actualiza con supabase.auth.onAuthStateChange
}
```

La decodificación en cliente **solo sirve para la UI**: la verificación de la firma y la autorización real las hace el servidor (PostgREST + **Row Level Security**).

B. Justificación de implementación en el proyecto

B.1 Árbol de rutas y servicios

```
src/app/core/
├── auth/auth.store.ts              ← estado de sesión (signals) + mapa de permisos
├── guards/auth.guard.ts            ← CanActivateFn: ¿hay sesión?
├── guards/permission.guard.ts      ← factoría CanMatchFn: ¿tiene el permiso?
├── guards/unsaved-changes.guard.ts ← CanDeactivateFn
├── interceptors/auth.interceptor.ts
└── directives/has-permission.directive.ts
```

```
// app.routes.ts
export const routes: Routes = [
  { path: '', redirectTo: 'productos', pathMatch: 'full' },
  { path: 'login', loadComponent: () => import('./pages/login/login.page').then(m => m.LoginPage) },
  { path: 'productos', canActivate: [authGuard],
    loadComponent: () => import('./pages/productos/productos.page').then(m => m.ProductosPage) },
  { path: 'productos/nuevo', canMatch: [permissionGuard('productos:crear')],
    canDeactivate: [unsavedChangesGuard],
    loadComponent: () => import('./pages/producto-form/producto-form.page').then(m => m.ProductoFormPage) },
  { path: 'pedidos', canActivate: [authGuard],
    loadComponent: () => import('./pages/pedidos/pedidos.page').then(m => m.PedidosPage) },
  { path: 'dashboard', canMatch: [permissionGuard('dashboard:ver')],
    loadComponent: () => import('./pages/dashboard/dashboard.page').then(m => m.DashboardPage) },
  { path: 'actividad', canMatch: [permissionGuard('actividad:ver')],
    loadComponent: () => import('./pages/actividad/actividad.page').then(m => m.ActividadPage) },
  { path: 'ayuda', loadComponent: () => import('./pages/ayuda/ayuda.page').then(m => m.AyudaPage) },
  { path: '**', redirectTo: 'productos' },
];
```

```
// permission.guard.ts — factoría: recibe el permiso y devuelve un guard configurado
export function permissionGuard(permiso: Permiso): CanMatchFn {
  return (_route, segments) => {
    const auth = inject(AuthStore);
    const router = inject(Router);
    const url = '/' + segments.map(s => s.path).join('/');  // canMatch no recibe RouterStateSnapshot
    if (!auth.isAuthenticated()) {
      return router.createUrlTree(['/login'], { queryParams: { returnUrl: url } });
    }
    return auth.can(permiso) ? true : router.createUrlTree(['/productos']);
  };
}
```

Redirección contextual: el `returnUrl` se conserva en la query; tras el login, `LoginPage` navega con `router.navigateByUrl(returnUrl ?? '/productos')` y, si esa URL no está permitida, el propio guard vuelve a reconducir. **Arranque:** la sesión de Supabase se restaura de forma asíncrona; `provideAppInitializer(() => inject(AuthStore).init())` garantiza que los guards no se evalúan antes de conocerla (evita expulsar al usuario al pulsar F5).

Jerarquía de roles y pantallas:

Pantalla / acción	Permiso	Administrador	Usuario estándar
Catálogo de productos	`productos:ver`	✓	✓
Alta/edición de producto	`productos:crear` / `editar`	✓	✗ (ruta no se carga)
Realizar pedido	`pedidos:crear`	✗	✓
Pedidos	`pedidos:ver_todos` / `ver_propios`	todos	solo los suyos (RLS)
Dashboard con gráficos	`dashboard:ver`	✓	✗
Exportar PDF / CSV	`informes:exportar`	✓	✗
Actividad en tiempo real	`actividad:ver`	✓	✗

Defensa en profundidad en el servidor (RLS): aunque se manipule el cliente, PostgreSQL filtra filas por `auth.uid()` y por el claim `app_rol`, p. ej. `create policy "pedidos propios" on pedidos for select using (usuario_id = auth.uid() or (auth.jwt() ->> 'app_rol') = 'admin');`

Registro del interceptor en la configuración global (`main.ts`), un único punto para toda la app:

```
bootstrapApplication(AppComponent, { providers: [
  { provide: RouteReuseStrategy, useClass: IonicRouteStrategy },
  provideIonicAngular(),
  provideRouter(routes, withComponentInputBinding()),            // sin PreloadAllModules (ver A.1)
  provideHttpClient(withInterceptors([authInterceptor])),
  provideAppInitializer(() => inject(AuthStore).init()),
]});
```

B.2 UI/UX condicional

Directiva estructural `*hasPermission`, reactiva gracias a `effect()` sobre la signal de permisos (si el rol cambia o se cierra sesión, el DOM se actualiza solo):

```
@Directive({ selector: '[hasPermission]' })
export class HasPermissionDirective {
  hasPermission = input.required<Permiso>();
  private tpl = inject(TemplateRef); private vcr = inject(ViewContainerRef);
  private auth = inject(AuthStore);
  constructor() {
    effect(() => {
      this.vcr.clear();
      if (this.auth.can(this.hasPermission())) this.vcr.createEmbeddedView(this.tpl);
    });
  }
}
```

```
<!-- Menú lateral (ion-menu): la entrada no existe en el DOM si no hay permiso -->
<ion-item *hasPermission="'dashboard:ver'" routerLink="/dashboard">Cuadro de mando</ion-item>
<ion-button *hasPermission="'informes:exportar'" (click)="exportarPdf()">Exportar PDF</ion-button>
```

Se elimina el elemento del DOM (no se oculta con CSS), lo que también lo retira del árbol de accesibilidad: el lector de pantalla no anuncia acciones inaccesibles (WCAG). Ocultar **no es seguridad**: es experiencia de usuario; la protección la aportan guard + RLS.

C. Matriz de alternativas evaluadas

Guards funcionales vs. comprobación en `ngOnInit()` / `ionViewWillEnter()`

Criterio	Guard funcional (elegido)	Comprobación en el componente
Seguridad	Decide antes de crear el componente; con `canMatch` el código ni se descarga	El componente ya se ha instanciado y renderizado un instante; su código está en el cliente
Sobrecarga de red	Nula para rutas sin permiso	Se descarga el chunk y puede lanzar peticiones antes de redirigir
Mantenibilidad	La política está en `app.routes.ts`, visible de un vistazo	Lógica repetida en cada página; fácil olvidarla en una nueva
Escalabilidad	Una factoría reutilizable por permiso	Crece linealmente con el número de vistas

HTTP Interceptor vs. cabeceras manuales

Criterio	Interceptor centralizado (elegido)	Cabeceras en cada servicio
Seguridad	Imposible olvidar el token; filtrado por dominio para no filtrarlo a terceros	Un servicio olvidado envía peticiones anónimas o filtra el token
Errores 401/403	Tratamiento único (refresh + reintento + logout)	Cada llamada duplica `catchError`
Mantenibilidad	Cambiar el esquema (p. ej. otro header) = 1 archivo	Cambiar N servicios
Testabilidad	Se prueba aislado con `HttpTestingController`	Hay que probarlo en cada servicio

RBAC vs. ABAC vs. flag booleano `isAdmin`

Criterio	Flag `isAdmin`	RBAC con permisos (elegido)	ABAC
Expresividad	2 estados fijos	Roles → conjuntos de permisos	Reglas por atributos (dueño, hora, importe…)
Escalabilidad	Un tercer rol obliga a reescribir guards y plantillas	Nuevo rol = una línea en `PERMISOS_POR_ROL`	Muy alta, pero cada regla es código
Mantenibilidad	Condiciones `if (isAdmin)` dispersas	Permisos con nombre, tipados (`Permiso`)	Motor de políticas complejo de auditar
Encaje en el proyecto	Insuficiente (rúbrica)	2 roles y 9 permisos: escala adecuada	Sobredimensionado; el único atributo necesario ("pedidos propios") ya lo cubre RLS con `auth.uid()`

Conclusión. La combinación *guards funcionales (`canMatch` en rutas restringidas) + interceptor centralizado + RBAC con permisos en un `AuthStore` de signals + RLS en PostgreSQL* separa autenticación de autorización, desacopla token, estado e interfaz, minimiza la carga de código no autorizado y mantiene la seguridad real en el servidor.